

CSP-Bender Problem

Formulation, Solution in AMPL, and Comparison with Heuristic Methods

Liliu Martinez

Enrique Takiguchi

Contents

1	Introduction	2
2	Problem Description and Formulation	2
2.1	Basic definitions	2
2.2	CSP Formulation	2
2.3	Benders decomposition	3
3	Datasets	6
3.1	small	6
3.2	example_2D	7
3.3	targus	7
4	Implementation Details	9
4.1	Subproblem Lower	9
4.2	Subproblem Upper	9
5	Computational Experiments and Results	9
5.1	small	9
5.2	Example_2D	9
5.3	targus	10
6	References	11
A	AMPL Files	12

1 Introduction

The Cell Supression Problem (CSP) is a fundamental problem in tabular data protection, as it must be solved to ensure a certain level of confidentiality is applied to specific sensible cells. Tabular data with unsolved CSP takes the risk that an attacker might be able to compute an interval for the values of sensible censured cells which is too small.

Since supressing only sensible cells would not suffice, the CSP seeks to find a balance between the amount of information and the protection of the tables, by defining minimum protection levels and the cost of hiding each cell.

As we will see, the complexity of the raw problem requires an algorithm to solve the problem in an efficient way, for which we will use Benders decomposition.

Objectives:

- Contextualizing the problem
- Formulating the problem
- Implementing CSP in its basic form (MILP) in AMPL
- Implementing CSP with Benders decomposition in AMPL
- Testing the implementations

2 Problem Description and Formulation

2.1 Basic definitions

- **Microdata:** a file V of s individuals and t variables, as a matrix $s \times t$ where V_{ij} is the value of variable j for individual i .
- **Tabular data:** tables generated crossing one ore more categorical variables from microdata. These can be multidimensional tables holding frequency or magnitude information. The table will have n cells which will have to verify m linear constraints. The cells in the table are bounded by an upper and lower limit: $u < a_i < l$

- **Protection levels from the p% rule:**

An attacker might estimate the intervals of a cell $[\bar{a}_p, \underline{a}_p]$, to assert these are not informative enough, we must define some lower and upper protection level bounds to ensure this interval fully contains the interval

$$[\bar{a}_p, \underline{a}_p] \in [a_p - lpl_p, a_p + upl_p] \quad (1)$$

The best guess an auditor can make about a_p is defined by $[\bar{a}_p, \underline{a}_p] \rightarrow$

$$\underline{a}_p = \begin{array}{ll} \min & x_p \\ \text{s.to} & Ax = b \\ & l_i \leq x_i \leq u_i \quad i \in \mathcal{P} \cup \mathcal{S} \\ & x_i = a_i \quad i \notin \mathcal{P} \cup \mathcal{S} \end{array} \quad \bar{a}_p = \begin{array}{ll} \max & x_p \\ \text{s.to} & Ax = b \\ & l_i \leq x_i \leq u_i \quad i \in \mathcal{P} \cup \mathcal{S} \\ & x_i = a_i \quad i \notin \mathcal{P} \cup \mathcal{S} \end{array} \quad (2)$$

2.2 CSP Formulation

The Cell Suppression Problem seeks to hide sensitive (primary, $\mathcal{P}$) cells while minimizing the suppression cost. Hiding only primary cells would not be enough since we could still infer information about them from the rest of cells, thus, a number of secondary cells ($\mathcal{S}$) will also be suppressed.

CSP computes the set $\mathcal{S}$ of secondary cells to be suppressed such that for all $p \in \mathcal{P}$, Equation 1 is satisfied, where $\bar{a}_p, \underline{a}_p$ can be obtained with a minimization problem.

The CSP problem can be formulated as a Mixed-Integer Linear Programming (MILP) problem. The MILP will be a minimization problem in terms of the following variables

- $y_i \in \{0, 1\}, i = 1, \dots, n$: Indicates if a cell i is suppressed (1) or not (0)
- $x^{l,p} \in \mathbb{R}^n$: Lower deviation of the primary cell p from its original value
- $x^{u,p} \in \mathbb{R}^n$: Upper deviation of the primary cell p from its original value

MILP formulation for CSP:

$$\begin{aligned}
& \min \quad \sum_{i=1}^n w_i y_i \\
& \text{s.to} \quad \left. \begin{aligned}
& Ax^{l,p} = 0 \\
& (l_i - a_i)y_i \leq x_i^{l,p} \leq (u_i - a_i)y_i \quad i = 1, \dots, n \\
& x_p^{l,p} \leq -lpl_p
\end{aligned} \right\} \forall p \in \mathcal{P} \\
& \left. \begin{aligned}
& Ax^{u,p} = 0 \\
& (l_i - a_i)y_i \leq x_i^{u,p} \leq (u_i - a_i)y_i \quad i = 1, \dots, n \\
& x_p^{u,p} \geq upl_p
\end{aligned} \right\} \\
& y_i \in \{0, 1\} \quad x \geq 0 \quad i = 1, \dots, n,
\end{aligned}$$

The problem with the MILP formulation is that it becomes infeasibly complex as the table grows large.

The problem has:

- n binary variables and $2n\mathcal{P}$ continuous variables
- $2(m + 2n)|\mathcal{P}|$ constraints

2.3 Benders decomposition

Benders decomposition partitions the problem into a master and subproblems, and iterates efficiently between them to find the optimal solution.

- Master problem: focuses on the binary decisions (y_i), which determine whether a cell is published ($y_i = 0$) or suppressed ($y_i = 1$)
- Subproblems: verify if the current suppression pattern chosen by the Master Problem actually provides the required protection levels (lpl and upl) for each primary sensitive cell.

Original Problem

Given the original problem:

$$\begin{aligned}
(P) = \quad & \min \quad c^T x + q^T y \\
& \text{s. to} \quad Tx + Wy = h \\
& x \in X \\
& y \geq 0
\end{aligned}$$

The two following problems are derivated:

- **Master Formulation**

$$\begin{aligned}
(BP_r) = \quad & \min \quad z \\
\text{s. to} \quad & z \geq c^T x + u^{i\top} (h - Tx) & i \in I \subseteq \{1 \dots p\} \\
& v^{j\top} (h - Tx) \leq 0 & j \in J \subseteq \{1 \dots q\} \\
& x \in X.
\end{aligned}$$

- **Subproblem Formulation**

$$\begin{aligned}
(Q_D) = \quad & \max \quad u^T (h - Tx) \\
\text{s. to} \quad & W^T u \leq q \\
& u \in \mathbb{R}^m
\end{aligned}$$

The Original Problem now becomes:

$$\begin{array}{ll}
\text{Objective function (min)} & \begin{array}{l} \mathbf{c}^T \mathbf{x} \quad \mathbf{q}^T \mathbf{y} \\ cy \end{array}
\end{array}$$

Constraint Type	Tx	+Wy	=h
Table Conservation	0	$+Ax^{l,p}$	$=0$
		$+Ax^{u,p}$	$=0$
Lower Capacity	$(l-a)y$	$-x^{l,p}$	≤ 0
	$(l-a)y$	$-x^{u,p}$	
Upper Capacity	$(a-u)y$	$+x^{l,p}$	≤ 0
	$(a-u)y$	$+x^{u,p}$	
Protection Levels	0	$x^{l,p}$	$\leq -lpl$
		$x^{u,p}$	$\geq upl$

$$\begin{array}{ll}
& x \in X \quad y \geq 0 \\
\text{Domain Constraints} & y_i \in \{0, 1\} \quad x, s \geq 0
\end{array}$$

Primal Subproblem Q: subproblem as an optimization to see if the protection level can be reached given the fixed suppression $\bar{y}$: for each primary cell $p[j]$:

$$\begin{aligned}
(Q_{lower}) = \quad & \min \quad x_{p[j]} \\
\text{s. to} \quad & Ax_l = 0[\lambda] \\
& (l_i - a_i)\bar{y}_i \leq x_{l,i}[\mu_{ul}] \\
& x_{l,i} \leq (u_i - a_i)\bar{y}_i[\mu_{ul}] \\
& \text{Goal : Objective value} \leq -lpl_{p[j]}?
\end{aligned}$$

$$\begin{aligned}
(Q_{upper}) = \quad & \max \quad x_{p[j]} \\
\text{s. to} \quad & Ax_u = 0[\lambda] \\
& (l_i - a_i)\bar{y}_i \leq x_{u,i}[\mu_{ul}] \\
& x_{u,i} \leq (u_i - a_i)\bar{y}_i[\mu_{ul}] \\
& \text{Goal : Objective value} \leq -upl_{p[j]}?
\end{aligned}$$

Dual of Subproblem Q: To ensure a suppression pattern is valid, you have to check if a primary cell is protected from "below" (someone trying to guess a value too low) and from "above" (someone trying to guess a value too high). Dual of Subproblem Q for each primary cell $p[j]$:

$$\begin{aligned} (Q_{Dlower}) = \quad & \max \quad \sum_{i=1}^n ((l_i - a_i)\mu_{ll,i} - (u_i - a_i)\mu_{ul,i})\bar{y}_i \\ \text{s. to} \quad & A^T\lambda + \mu_{ll} - \mu_{ul} = e_p \\ & \mu_{ll} \geq 0, \mu_{ul} \geq 0, \lambda \text{ free} \end{aligned}$$

If the optimal value of this objective is greater than $-lpl$, that means the pattern is insecure and we have to add a lower Benders cut to the Master Problem.

$$-lpl \geq \sum_{i=1}^n ((l_i - a_i)\mu_{ll,i} - (u_i - a_i)\mu_{ul,i})y_i$$

$$\begin{aligned} (Q_{Dupper}) = \quad & \max \quad \sum_{i=1}^n ((u_i - a_i)\mu_{uu,i} - (l_i - a_i)\mu_{lu,i})\bar{y}_i \\ \text{s. to} \quad & A^T\lambda + \mu_{lu} - \mu_{uu} = -e_p \\ & \mu_{lu} \geq 0, \mu_{uu} \geq 0, \lambda \text{ free} \end{aligned}$$

If the optimal value of this objective is less than upl , that means the pattern is insecure and we have to add an upper Benders cut to the Master Problem.

$$upl \leq \sum_{i=1}^n ((u_i - a_i)\mu_{uu,i} - (l_i - a_i)\mu_{lu,i})y_i$$

If we have P primary cells, we will have to solve $2P$ subproblems at each iteration of Benders decomposition.

Master Problem: We have to add the cuts for all the insecure primary cells found in the subproblems. If we have

- $\mathcal{J}_l$: set of lower cuts added until iteration r from insecure primary cells found with the lower dual subproblem
- $\mathcal{J}_u$: set of upper cuts added until iteration r from insecure primary cells found with the upper dual subproblem
- for each cut $j_l \in \mathcal{J}_l$, we have $\bar{\mu}_{ll}^{p,j_l}, \bar{\mu}_{ul}^{p,j_l}$ from the solution of the corresponding lower dual subproblem
- for each cut $j_u \in \mathcal{J}_u$, we have $\bar{\mu}_{lu}^{p,j_u}, \bar{\mu}_{uu}^{p,j_u}$ from the solution of the corresponding upper dual subproblem

The Master Problem at iteration r is:

$$\begin{aligned}
(BP_r) = \quad & \min \sum_{i=1}^n c_i y_i \\
\text{s. to} \quad & y_p = 1 \quad \forall p \in \mathcal{P} \\
& -lpl_p \geq \sum_{i=1}^n ((l_i - a_i) \bar{\mu}_{ll,i}^{p,j_l} - (u_i - a_i) \bar{\mu}_{ul,i}^{p,j_l}) y_i \quad \forall p \in \mathcal{P}, \forall j_l \in \mathcal{J}_l \\
& upl_p \leq \sum_{i=1}^n ((u_i - a_i) \bar{\mu}_{uu,i}^{p,j_u} - (l_i - a_i) \bar{\mu}_{lu,i}^{p,j_u}) y_i \quad \forall p \in \mathcal{P}, \forall j_u \in \mathcal{J}_u \\
& y_i \in \{0, 1\}
\end{aligned}$$

3 Datasets

3.1 small

Original table: In Table 1 we can see the original data table for *small.ampl* data instance created by rows($\rightarrow$), with the sensitive cells marked with *. Column 7 and Row 5 contain the totals for the rows and columns respectively.

	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7
Row 1	300*	8	5	5	5*	38	361
Row 2	8	68*	40	76	29	31	252
Row 3	11	33	20	60	35	44	203
Row 4	7	28	36*	41	22	63	197
Row 5	326	137	101	182	91	176	1013

Table 1: Original Small Instance Data Table with Sensitive Cells Marked with *

Number Cell	Lower Protection Level	Upper Protection Level
1	15	20
5	2	4
9	10	6
24	9	3

Table 2: Table of Small Instance Protection Levels for Sensitive Cells

Table Constraints:

Row Constraints:

1. $x_1 + x_2 + x_3 + x_4 + x_5 + x_6 - x_7 = 0$
2. $x_8 + x_9 + x_{10} + x_{11} + x_{12} + x_{13} - x_{14} = 0$
3. $x_{15} + x_{16} + x_{17} + x_{18} + x_{19} + x_{20} - x_{21} = 0$
4. $x_{22} + x_{23} + x_{24} + x_{25} + x_{26} + x_{27} - x_{28} = 0$

Column Constraints:

5. $x_1 + x_8 + x_{15} + x_{22} - x_{29} = 0$

6. $x_2 + x_9 + x_{16} + x_{23} - x_{30} = 0$
7. $x_3 + x_{10} + x_{17} + x_{24} - x_{31} = 0$
8. $x_4 + x_{11} + x_{18} + x_{25} - x_{32} = 0$
9. $x_5 + x_{12} + x_{19} + x_{26} - x_{33} = 0$
10. $x_6 + x_{13} + x_{20} + x_{27} - x_{34} = 0$

3.2 example_2D

In Table 3 we can see the original data table for *example_2D.ampl* data instance created by rows($\rightarrow$), with the sensitive cells marked with *. Column 1 and Row 1 represent the totals for their respective dimensions.

	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6
Row 1	3220	84	632	930	843	731
Row 2	1529	3	336	309	484	397
Row 3	484	25	3	393*	48	15
Row 4	392	1	2	137*	145	107
Row 5	815	55	291*	91	166	212*

Table 3: Original Example_2D Instance Data Table with Sensitive Cells Marked with *

Number Cell	Lower Protection Level	Upper Protection Level
16	40	30
22	14	14
27	15	30
30	21	21

Table 4: Table of Example_2D Instance Protection Levels for Sensitive Cells

3.3 targus

In Table 5 we can see the original data table for *targus.ampl* data instance created by rows($\rightarrow$), with the sensitive cells marked with *. Due to the large values in the table, we have used commas to separate thousands. Due to the row-wise size of the table, we have transposed the original table for better visualization. Row 1 contains the totals for the columns.

	Row 1	Row 2	Row 3	Row 4	Row 5	Row 6	Row 7	Row 8	Row 9
Col 1	16847261.84	20*	25	2711808	2320534	2505042.58	2799074.26	6510758	0
Col 2	4373279	5*	5*	719049	659680	688962	756529	1549049	0
Col 3	1986129	5*	5*	398062	348039	354711	418778	466529	0
Col 4	1808861	0	0	223990	221332	241913	258233	863393	0
Col 5	578289	0	0	96997	90309	92338	79518	219127	0
Col 6	3703896	15*	5*	642238	515003	534147	620392	1392096	0
Col 7	124336	5*	0	36311	32132	25770	18150	11968*	0
Col 8	526279	0	0	93589	94957	110930	81799	145004	0
Col 9	2234995	10*	5*	345803	251358	251188	303377	1083254	0
Col 10	818286	0	0	166535	136556	146259	217066	151870	0
Col 11	4576115.84	0	0	648972	543570	663896.58	775132.26	1944545	0
Col 12	485326	0	0	63767	75442	87305	59953	198859	0
Col 13	3664559.84	0	0	537911	430851	515019.58	643762.26	1537016	0
Col 14	426230	0	0	47294	37277	61572	71417	208670	0
Col 15	4193971	0	15*	701549	602281	618037	647021	1625068	0
Col 16	2752743	0	15*	488613	392395	363490	402925	1105305	0
Col 17	1441228	0	0	212936	209886	254547	244096	519763	0
Col 18	0	0	0	0	0	0	0	0	0

Table 5: Original Targus Instance Data Table with Sensitive Cells Marked with *

Number Cell	Lower Protection Level	Upper Protection Level
19	6	6
20	1.67	1.67
21	1.67	1.67
24	5	5
25	1.67	1.67
27	3.33	3.33
38	1.67	1.67
39	1.67	1.67
42	1.67	1.67
45	1.67	1.67
51	5	5
52	5	5
133	3480	3480

Table 6: Table of Targus Instance Protection Levels for Sensitive Cells

4 Implementation Details

4.1 Subproblem Lower

4.2 Subproblem Upper

5 Computational Experiments and Results

5.1 small

Cell ID	Cost	Is Primary
1	1	1
3	1	0
5	1	1
8	1	0
9	1	1
10	1	0
12	1	0
15	1	0
22	1	0
24	1	1

Table 7: Table of Cells to be Suppressed in Small Instance

	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7
Row 1	x	8	x	5	x	38	361
Row 2	x	x	x	76	x	31	252
Row 3	x	33	20	60	35	44	203
Row 4	x	28	x	41	22	63	197
Row 5	326	137	101	182	91	176	1013

Table 8: Small Data Table with Suppressing Sensitive Cells

5.2 Example_2D

Cell ID	Cost	Is Primary
3	0	0
6	0	0
13	0	0
16	0	1
19	0	0
22	0	1
27	0	1
30	0	1

Table 9: Table of Cells to be Suppressed in example_2D Instance

	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6
Row 1	3220	84	x	930	843	x
Row 2	1529	3	336	309	484	397
Row 3	x	25	3	x	48	15
Row 4	x	1	2	x	145	107
Row 5	815	55	x	91	166	x

Table 10: example_2D Data Table with Suppressing Sensitive Cells

5.3 targus

Cell ID	Cost	Is Primary
19	1	1
20	1	1
21	1	1
24	1	1
25	1	1
27	1	1
37	1	0
38	1	1
39	1	1
42	1	1
45	1	1
51	1	1
52	1	1
78	611	0
87	715	0
99	298	0
106	432	0
133	14	1
134	172	0

Table 11: Table of Cells to be Suppressed in Targus Instance

	Row 1	Row 2	Row 3	Row 4	Row 5	Row 6	Row 7	Row 8	Row 9
Col 1	16847261.84	x	x	2711808	2320534	2505042.58	2799074.26	6510758	0
Col 2	4373279	x	x	719049	659680	688962	756529	1549049	0
Col 3	1986129	x	x	398062	348039	354711	418778	466529	0
Col 4	1808861	0	0	223990	221332	241913	258233	863393	0
Col 5	578289	0	0	96997	90309	92338	79518	219127	0
Col 6	3703896	x	x	642238	x	534147	620392	1392096	0
Col 7	124336	x	0	36311	32132	25770	18150	x	0
Col 8	526279	0	0	93589	94957	110930	81799	x	0
Col 9	2234995	x	x	345803	251358	x	303377	1083254	0
Col 10	818286	0	0	166535	136556	146259	217066	151870	0
Col 11	4576115.84	0	0	648972	543570	663896.58	775132.26	1944545	0
Col 12	485326	0	0	63767	75442	87305	59953	198859	0
Col 13	3664559.84	0	0	537911	430851	515019.58	643762.26	1537016	0
Col 14	426230	0	0	47294	37277	61572	71417	208670	0
Col 15	4193971	0	x	701549	x	618037	647021	1625068	0
Col 16	2752743	0	x	488613	392395	x	402925	1105305	0
Col 17	1441228	0	0	212936	209886	254547	244096	519763	0
Col 18	0	0	0	0	0	0	0	0	0

Table 12: Targus Data Table with Suppressing Sensitive Cells

6 References

A AMPL Files

```
1 param n; # observations
2 param k; # clusters
3 set M:={1..n};
4
5 param d{M,M};
6 var x{M,M} binary;
7
8
9 minimize f: sum{i in M}
10      sum{j in M} d[i,j]*x[i,j];
11
12 subject to one_cluster{i in M}:
13      sum{j in M} x[i,j] = 1;
14
15 subject to k_clusters:
16      sum{j in M} x[j,j] = k;
17
18 subject to cluster_belongs{i in M, j in M}:
19      x[j,j] >= x[i,j];
```

Listing 1: kmedoid.mod

```
1 # Reset enviroment
2 reset;
3
4 # Load model and data
5 model kmedoids.mod;
6 data kmedoids.dat;
7
8 option solver cplex;
9 solve;
10
11 display _solve_elapsed_time;
12
13 printf {i in M, j in M} "%s,%s,%i\n", i, j, x[i,j] > "x_matrix.csv";
```

Listing 2: kmedoid.run

```
1 def generate_dat(N, D, k, filename='AMPL/kmedoid.dat'):
2     with open(filename, 'w') as f:
3         f.write("#Define sets\n")
4         f.write(f"param n := {N};\n")
5         f.write(f"param k := {k};\n")
6
7         f.write("#Define parameters\n")
8         f.write(f"param d: {'\t'.join([str(j+1) for j in range(N)])} :=\n")
9         for i in range(N):
10             f.write(f" {i+1} " + ' '.join(f"{D[i][j]:.6f}" for j in range(N)) +
11                 "\n")
12             f.write(";\n")
13
```

Listing 3: Python function to generate kmedoid.dat